



Print(Null);



FASE INICIACIÓN



Sobre nosotros



`print(Null);`



Leidy Giraldo

Líder de equipo



Santiago Herrada

Líder de planeación



Esteban Prieto

Líder de calidad



Nicolás Cubillos

Líder de soporte



Kerlon Gutiérrez

Líder de desarrollo



Matías Marín

Líder de arquitectura



PROYECTO



DashKPI

DashKPI es una plataforma web diseñada para registrar, medir y visualizar indicadores clave de rendimiento (KPIs) de proyectos en tiempo real. Permitirá a los equipos gestionar tareas, monitorear avances y tomar decisiones basadas en datos mediante un dashboard interactivo.

Objetivo principal

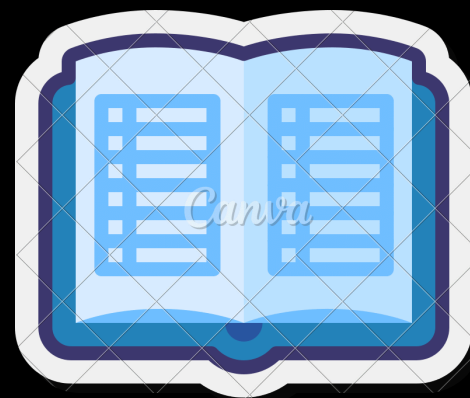
Desarrollar una herramienta que facilite el seguimiento del progreso, la calidad y el uso de recursos en proyectos, optimizando la gestión y el cumplimiento de metas.

Alcance

Incluye registro de proyectos y tareas, cálculo automático de KPIs, visualización gráfica, generación de reportes y notificaciones de alertas. No contempla el desarrollo de aplicación móvil en esta fase.



Bitácoras



- ¿Qué son? Registros de tiempo y tareas por persona (fecha, fase, un comentario) para medir esfuerzo real vs. estimado.
- Qué hicimos:
 - Planeación inicial, definición de alcance y metas, organización de KPIs por rol, redacción del acta.
 - Diseño UI (paleta, tipografía, layout del dashboard), creación/revisión del logo y del sitio web.
 - Revisión y corrección de documentos para la entrega.
- Quiénes y focos:
 - Leidy (Líder): coordinación, alcance, KPIs, acta, diseño UI.
 - Santiago (Planeación): asignación de tareas, agenda, control de asignaciones.
 - Matías (Arquitectura): apoyo en alcance/KPIs y revisión documental.
 - Esteban (Calidad): documentación, logo, “script de procesos”, orden de entregables.
 - Nicolás (Soporte): logo, ajustes del sitio, consolidación documental.
 - Kerlon(Desarrollo):apoyo en alcance/KPIs y revisión documental.
- Para qué sirve: evidencia de avance, base para estimar mejor y detectar cuellos de botella.



`print(Null);`



Log de defectos (Fase 1)

¿Qué es?

Historial de errores hallados y corregidos con: tipo, fase de inyección/remoción, tiempo de resolución y descripción.

Defectos relevantes resueltos

Requerimiento mal definido: objetivo demasiado ambicioso; se ajustó al alcance real.

Defectos relevantes resueltos

Redacción/incongruencia en alcance: “sin móvil ahora, pero tal vez a futuro”; se dejó explícito: móvil no incluido en esta fase.

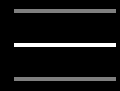
Defectos relevantes resueltos

Error de diseño de logo: no reflejaba identidad; calidad y soporte iteraron hasta cumplir criterios.

Defectos relevantes resueltos

Script mal enfocado: se rehizo alineado al acuerdo de la primera reunión.





Acta de iniciación (puntos clave)



`print(Null);`

- Equipo: Print(Null); roles definidos (líder, planeación, calidad, soporte, desarrollo, arquitectura).
- Comunicación: WhatsApp (rápido), Discord (colaboración en vivo), Teams (gestión formal y repositorio).
- Entregas: GitHub con ramas y PRs; decisiones por voto técnico y validación del líder.
- Objetivo: construir un dashboard que mida progreso, calidad y recursos de proyectos.
- Alcance (incluye): auth/usuarios, proyectos/tareas, KPIs automáticos, dashboard con filtros, reportes (PDF/Excel), alertas.
- No incluye: móvil nativo ni integraciones ERP/pagos.
- Metas: MVP, $\geq 90\%$ de requisitos cumplidos, $\leq 2s$ respuesta, $\geq 85\%$ satisfacción, documentación completa.



OTROS DOCUMENTOS



Control de asignaciones

- ¿Qué es? Matriz para cada actividad con tamaño/tiempo estimado vs. real, fecha/hora de entrega y diferencia.
- Cómo lo usamos: registramos tamaño real (p. ej., hojas/secciones) y tiempo real (min).
- Diferencia = tiempo real – estimado → positivo (retraso), negativo (adelanto).
- Valor: disciplina de seguimiento, evidencias para ajustes de carga y planificación del siguiente ciclo.

Maestro de documentos

- ¿Qué es? Inventario central de artefactos: nombre, fase, fecha de creación/modificación, versión y URL.
- Estado actual (Fase 1): bitácoras por integrante, log de defectos, acta de reunión, acta de iniciación, control de asignaciones e informe.
- Valor: control de versiones y acceso rápido; evita trabajar con documentos desactualizados.

Actas de reunión (Ciclo 1)

- ¿Qué son? Minutas con temas tratados, acuerdos, responsables y fechas.
- Acuerdos clave: confirmación de roles, backlog inicial y prioridades, criterios de calidad, próxima reunión de control, responsables por documento/artefacto.
- Valor: memoria de decisiones y seguimiento de compromisos.



FASE ESTRATEGIA



DEFINICIÓN DE ESTRATEGIA



Objetivo

Desarrollar una plataforma web para registrar, medir y visualizar KPIs de proyectos, con un enfoque en calidad, usabilidad y eficiencia.

Criterios: Precisión en los datos, interfaz intuitiva, seguridad, escalabilidad y cumplimiento de tiempos y costos.

Estrategia General: Aplicar metodologías ágiles, entregar módulos funcionales en ciclos cortos y mantener la colaboración entre equipo y usuarios clave.

Diseño Conceptual

- La plataforma se compone de módulos de autenticación, proyectos/KPIs, registro de datos, dashboard, reportes, alertas, configuración y auditoría.
- Funcionalidades Clave: Login con roles, CRUD de proyectos y KPIs, carga de datos manual/CSV, dashboard con filtros, reportes exportables, alertas por umbrales y bitácora de cambios.

Estimación Preliminar (Ciclo 1):

- Total aproximado: 1.340 líneas de código
- Esfuerzo estimado: 90 horas de desarrollo





print(**Null**);

Plan de administración de configuración



- La administración de configuración en DashKPI asegura la identificación, control y mantenimiento de los elementos del software, garantizando su integridad y trazabilidad. El plan define roles claros para líderes de equipo, desarrollo, arquitectura, soporte, calidad y planificación, quienes gestionan cambios, revisiones y auditorías. Se apoyará en herramientas como Git, Jira y repositorios documentales, junto con políticas de respaldo y capacitación. Con ello se garantiza un proceso sostenible que mantiene la calidad, estabilidad y escalabilidad del sistema.



Gestión de riesgos



`print(Null);`

- Definimos criterios de valoración:
 - Se establecieron escalas del 1 al 10 para medir la probabilidad, el impacto y el costo de retiro de cada riesgo.
 - Se determinó la fórmula para calcular el peso ($P_s = P_b \times I_m \times C_r$) y con él clasificar los riesgos en Alta, Media o Baja prioridad.
- Identificamos riesgos clave:
 - Se listaron 12 riesgos principales relacionados con el proyecto: retrasos en entregas, cambios en requisitos, errores en fuentes de datos, bajo rendimiento del sistema y problemas de comunicación.
 - Cada riesgo fue codificado (RG-001, RG-002, etc.), descrito y valorado con números justificados en probabilidad, impacto y costo de retiro.
- Planificamos acciones de mitigación y contingencia:
 - Para cada riesgo se diseñaron acciones preventivas (mitigación) y planes de respuesta en caso de ocurrencia (contingencia).
 - Además, se asignaron responsables según los roles TSP (líder de planeación, calidad, arquitectura, etc.).
- Control de cambios:
 - Se incluyó una tabla para registrar modificaciones en el proyecto (qué cambio se pidió, quién lo aprobó, impacto y estado).
 - Esto asegura trazabilidad y evita confusiones durante la ejecución.



Log de defectos (Fase 2)

Defectos relevantes resueltos

Gestión de riesgos:
faltaba “Actividades de
seguimiento”; se añadió
y corrigieron
inconsistencias.

Defectos relevantes resueltos

Administración de
configuración: sección
desordenada; se
reorganizó y se alineó al
índice.

Defectos relevantes resueltos

Bitácoras: envíos
tardíos/olvidos; se
centralizó el registro y
se activaron
recordatorios.





OTROS DOCUMENTOS



Control de asignaciones

- ¿Qué es? Matriz para cada actividad con tamaño/tiempo estimado vs. real, fecha/hora de entrega y diferencia.
- Cómo lo usamos: registramos tamaño real (p. ej., hojas/secciones) y tiempo real (min).
- Diferencia = tiempo real – estimado → positivo (retraso), negativo (adelanto).
- Valor: disciplina de seguimiento, evidencias para ajustes de carga y planificación del siguiente ciclo.

Maestro de documentos

- ¿Qué es? Inventario central de artefactos: nombre, fase, fecha de creación/modificación, versión y URL.
- Estado actual (Fase 2): bitácoras por integrante, log de defectos, acta de reunión, definición de estrategia, gestión de riesgos, plan de admin de configuración, control de asignaciones e informe.
- Valor: control de versiones y acceso rápido; evita trabajar con documentos desactualizados.

Actas de reunión (Ciclo 1)

- ¿Qué son? Minutas con temas tratados, acuerdos, responsables y fechas.
- Acuerdos clave: confirmación de roles, backlog inicial y prioridades, criterios de calidad, próxima reunión de control, responsables por documento/artefacto.
- Valor: memoria de decisiones y seguimiento de compromisos.



FASE REQUERIMIENTOS



MODELO DE CASOS DE USO DEL SISTEMA DASHKPI



En el modelo de casos de uso para el sistema DashKPI, se identificaron y definieron 18 casos de uso fundamentales que cubren todas las interacciones clave entre los usuarios y el sistema. Estos casos de uso son representaciones de cómo los diferentes actores del sistema interactúan con la plataforma para lograr tareas específicas.

- Autenticación MFA: Gestión de seguridad con múltiples factores de autenticación.
- Gestionar roles y permisos: Asignación de roles y gestión de permisos de usuarios.
- Crear proyecto: Creación y configuración de nuevos proyectos en DashKPI.
- Planificar sprint / iteración: Definición y organización de los sprints del proyecto.
- Gestionar backlog: Administración de tareas e historias de usuario en el backlog.
- Registrar tiempo y avance: Registro del tiempo trabajado y seguimiento del progreso.
- Configurar y calcular KPIs: Definición y cálculo de indicadores clave de desempeño.
- Visualizar dashboard: Visualización de datos y KPIs en el tablero de control. Generar reportes y exportaciones: Creación de informes y exportación de datos.
- Registrar y gestionar riesgos: Identificación, evaluación y seguimiento de riesgos del proyecto.
- Log de defectos: Registro, seguimiento y cierre de defectos.





MODELO DE CASOS DE USO DEL SISTEMA DASHKPI



- Bitácoras y actas: Documentación y registro de actividades y decisiones.
- Aprobación de cambios: Control y validación de cambios en el proyecto.
- Notificaciones y alertas: Envío de notificaciones y alertas importantes.
- Integración con repositorios: Integración con sistemas como GitHub/GitLab.
- Gestión de usuarios y equipo: Administración de los usuarios y el equipo de trabajo.
- Maestro de documentos del proyecto: Gestión centralizada de los documentos del proyecto.
- Cerrar proyecto: Procedimiento para la finalización y cierre de un proyecto.

Cada uno de estos casos de uso está diseñado para mejorar la eficiencia del sistema, asegurando que todas las funcionalidades se alineen con las necesidades del equipo y los usuarios.

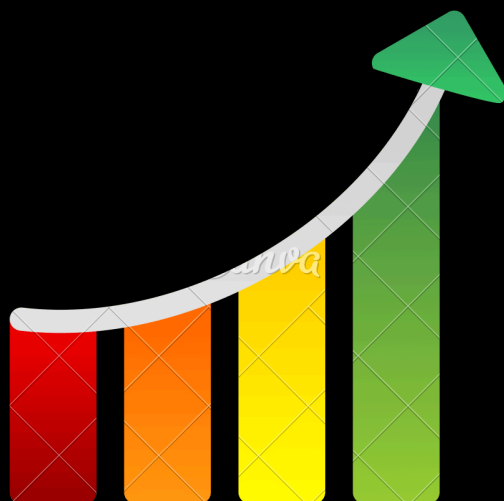
Relaciones y Extensiones:

- Algunos casos de uso tienen relaciones de include (un caso que siempre incluye otro) y extend (un caso que se ejecuta bajo ciertas condiciones, como notificaciones cuando ocurren ciertos eventos).

Este modelo es esencial para comprender cómo el sistema interactúa con los diferentes actores y garantiza que todas las funcionalidades estén bien definidas y sean eficientes.



Atributos de calidad



```
print(Null);
```

1. Seguridad

- La seguridad se centra en la protección de los datos críticos del sistema DashKPI, como KPIs, reportes y documentación. Para garantizar la confidencialidad, integridad y disponibilidad de la información, se implementaron controles de seguridad como autenticación multifactor (2FA), cifrado de datos y el uso de roles definidos para limitar el acceso.

KPIs relacionados:

- Tasa de intentos de intrusión bloqueados $\geq 99\%$
- Registro de intentos fallidos de acceso (100% de registros)

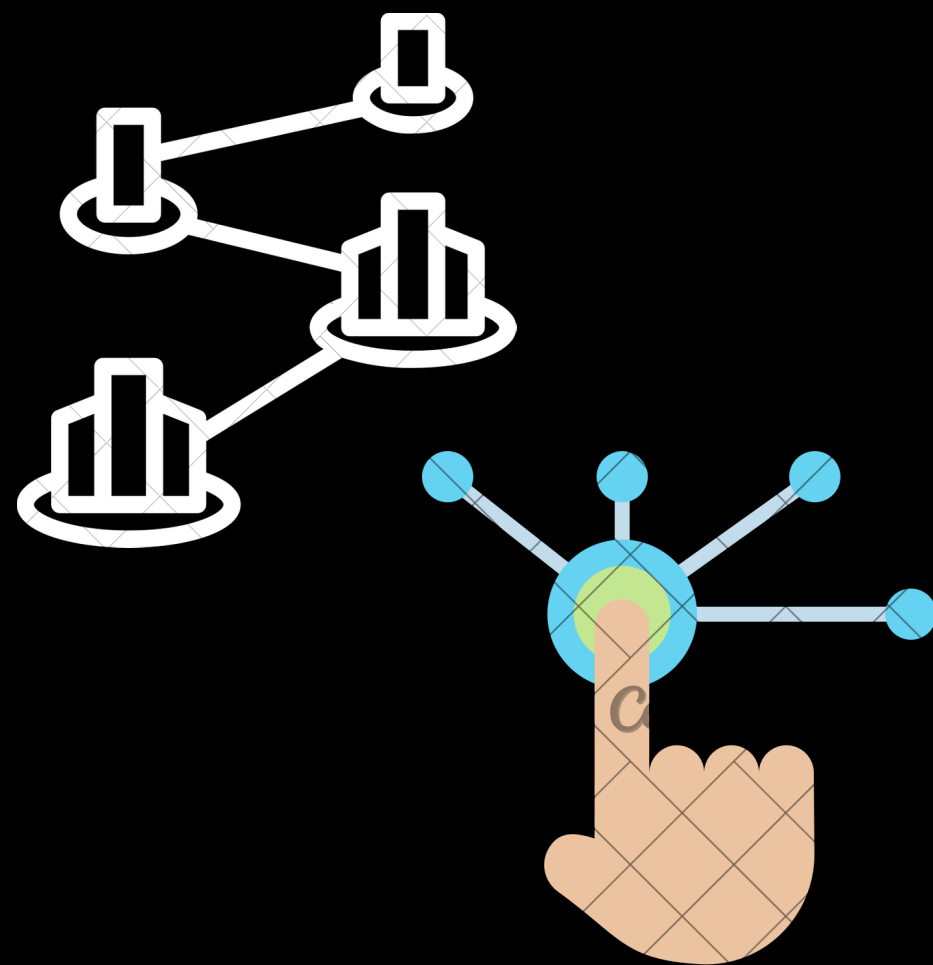
2. Desempeño

- El desempeño se asegura de que el sistema responda rápidamente, incluso en situaciones de alta demanda. Para ello, DashKPI optimiza la consulta de base de datos, utiliza caché para resultados recientes y balancea la carga entre servidores.

KPIs relacionados:

- Tiempo de respuesta de dashboards ≤ 3 segundos
- Tiempo de exportación de reportes ≤ 5 segundos
- Uso de CPU $\leq 70\%$ durante picos de carga

Atributos de calidad



```
print(Null);
```

3. Escalabilidad

La escalabilidad es crítica para permitir que DashKPI crezca con las necesidades del cliente. Esto significa que el sistema debe poder agregar nuevos módulos o integraciones sin necesidad de cambios en el código base del sistema.

KPIs relacionados:

- Tiempo de desarrollo para integrar nuevos conectores ≤ 5 días
- Costo de implementación del nuevo módulo $\leq 10\%$ del costo total inicial
- Número de líneas de código modificadas en el núcleo: 0

4. Usabilidad

La usabilidad se enfoca en la facilidad de uso del sistema. En el caso de la creación de KPIs, se implementaron validaciones en tiempo real y mensajes de error claros para evitar frustraciones por errores en las fórmulas.

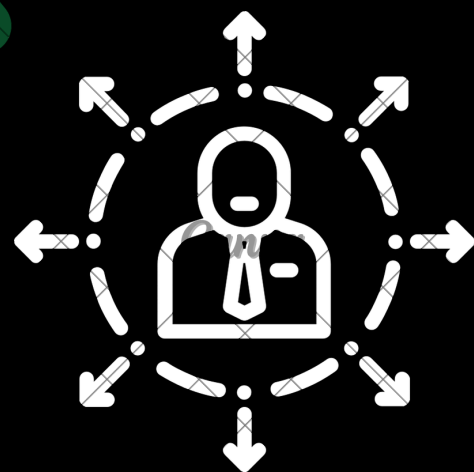
KPIs relacionados:

- Tiempo de respuesta para validaciones ≤ 0.5 segundos
- Tasa de recuperación sin pérdida de datos $\geq 99\%$
- Tasa de errores irreversibles = 0



print(Null);

Atributos de calidad



5. Disponibilidad

La disponibilidad garantiza que DashKPI esté disponible en todo momento, con tiempos de recuperación rápidos en caso de falla. Se implementaron mecanismos de alta disponibilidad y failover para asegurar que los usuarios puedan acceder a los dashboards y reportes sin interrupciones.

KPIs relacionados:

- Disponibilidad mensual $\geq 99.5\%$
- Tiempo máximo de recuperación tras una falla crítica ≤ 15 minutos
- Tiempo medio entre fallas (MTBF) ≥ 200 horas

6. Extensibilidad

La extensibilidad asegura que el sistema pueda integrar nuevos módulos o fuentes de datos sin afectar su arquitectura base. Esto permite que DashKPI se adapte fácilmente a las necesidades del cliente y crezca con el tiempo.

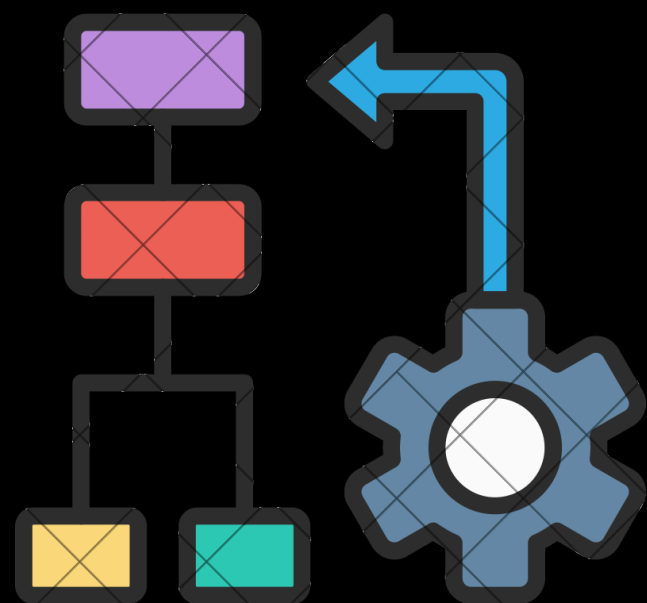
KPIs relacionados:

- Tiempo de desarrollo para nuevos conectores ≤ 5 días
- Costo de implementación del nuevo módulo $\leq 10\%$ del costo total inicial
- Tiempo de validación de la integración ≤ 2 días, incluyendo pruebas de regresión



```
print(Null);
```

Modelo de dominio



- El modelo de dominio de DashKPI permite a los usuarios gestionar y visualizar los KPIs de los proyectos que supervisan. Cada KPI se calcula mediante fórmulas que pueden involucrar datos históricos o referencias a otros KPIs. Los reportes y las notificaciones son parte integral del sistema, ayudando a los usuarios a mantenerse informados sobre el desempeño de sus proyectos.
- Este modelo está diseñado para ser extensible (permitiendo agregar nuevos KPIs, proyectos o funcionalidades) y debe soportar una alta disponibilidad y seguridad para los datos clave de los usuarios y proyectos.



Log de defectos (Fase 3)

Defectos relevantes resueltos

Hubo un retraso en la entrega de casos de uso, atributos de calidad y modelo de dominio debido a problemas de coordinación en el equipo.

Defectos relevantes resueltos

Algunos casos de uso no fueron recordados correctamente, lo que llevó a la necesidad de rehacerlos.

Defectos relevantes resueltos

Se cometieron errores en la redacción de los atributos de calidad, lo que afectó la claridad del documento.

Defectos relevantes resueltos

Hubo dificultades para crear gráficas en el control de asignaciones, lo que generó retrasos en su implementación.





OTROS DOCUMENTOS

Control de asignaciones

- ¿Qué es? Matriz para cada actividad con tamaño/tiempo estimado vs. real, fecha/hora de entrega y diferencia.
- Cómo lo usamos: registramos tamaño real (p. ej., hojas/secciones) y tiempo real (min).
- Diferencia = tiempo real – estimado → positivo (retraso), negativo (adelanto).
- Valor: disciplina de seguimiento, evidencias para ajustes de carga y planificación del siguiente ciclo.

Maestro de documentos y SRS

- Es un repositorio centralizado que contiene los enlaces a todos los documentos del proyecto, como casos de uso, atributos de calidad y modelos.
- SRS (Software Requirements Specification):
- Es el documento de soporte que recoge los requisitos del software, incluyendo los casos de uso, atributos de calidad y modelos, detallando cómo debe funcionar el sistema.

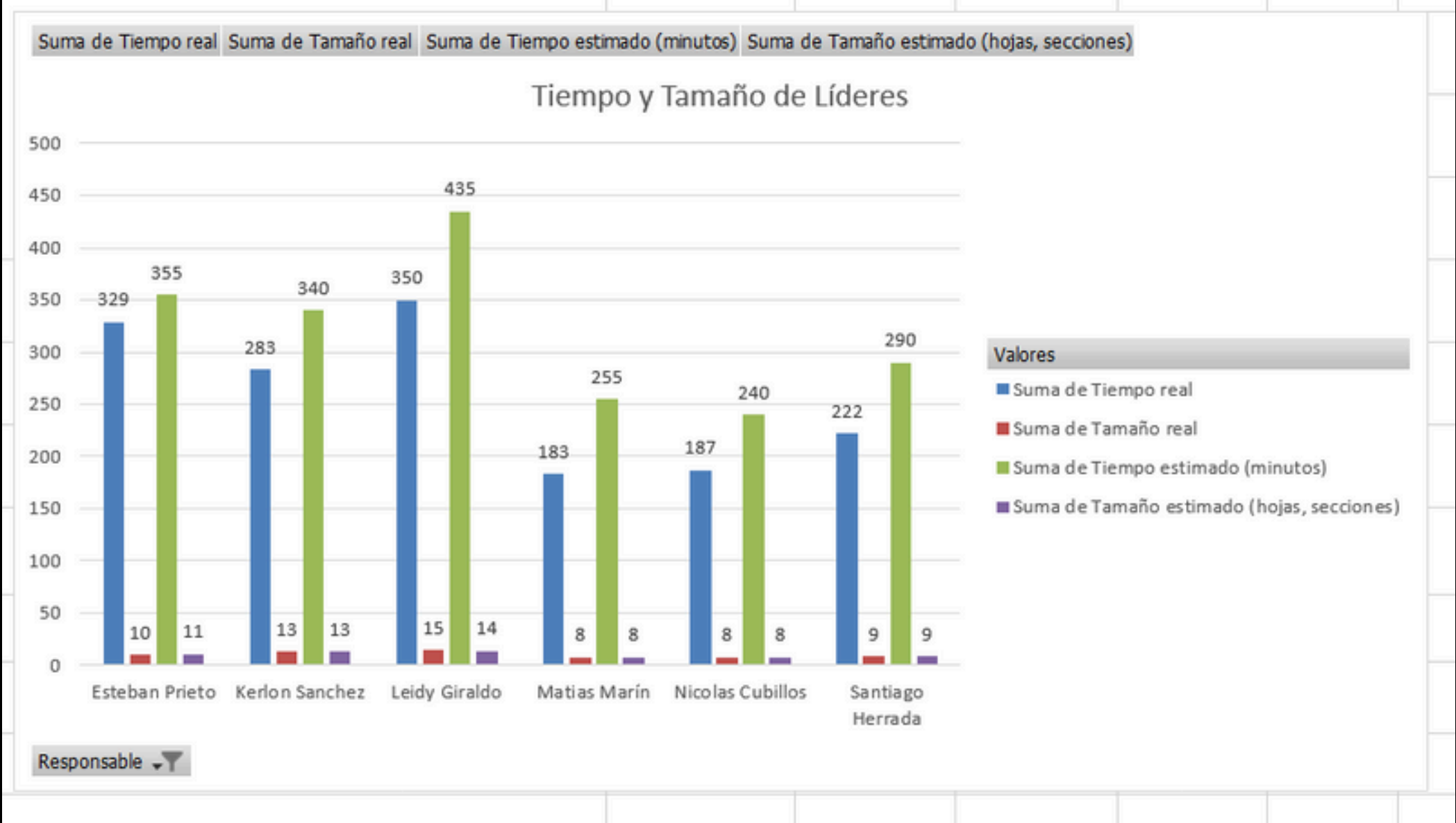
Actas de reunión (Ciclo 1)

- ¿Qué son? Minutas con temas tratados, acuerdos, responsables y fechas.
- Acuerdos clave: confirmación de roles, backlog inicial y prioridades, criterios de calidad, próxima reunión de control, responsables por documento/artefacto.
- Valor: memoria de decisiones y seguimiento de compromisos.



print(**Null**);

ESTADÍSTICAS





print(**Null**);

ESTADÍSTICAS

Etiquetas de fila ☐	Suma de Tiempo real	Suma de Tamaño real	Suma de Tiempo estimado (minutos)	Suma de Tamaño estimado (hojas, secciones)
Esteban Prieto	329	10	355	11
Kerlon Sanchez	283	13	340	13
Leidy Giraldo	350	15	435	14
Matias Marin	183	8	255	8
Nicolas Cubillos	187	8	240	8
Santiago Herrada	222	9	290	9
Total general	1554	63	1915	63



FASE PLANEACIÓN



PLAN DE ADMINISTRACIÓN DE REQUERIMIENTOS



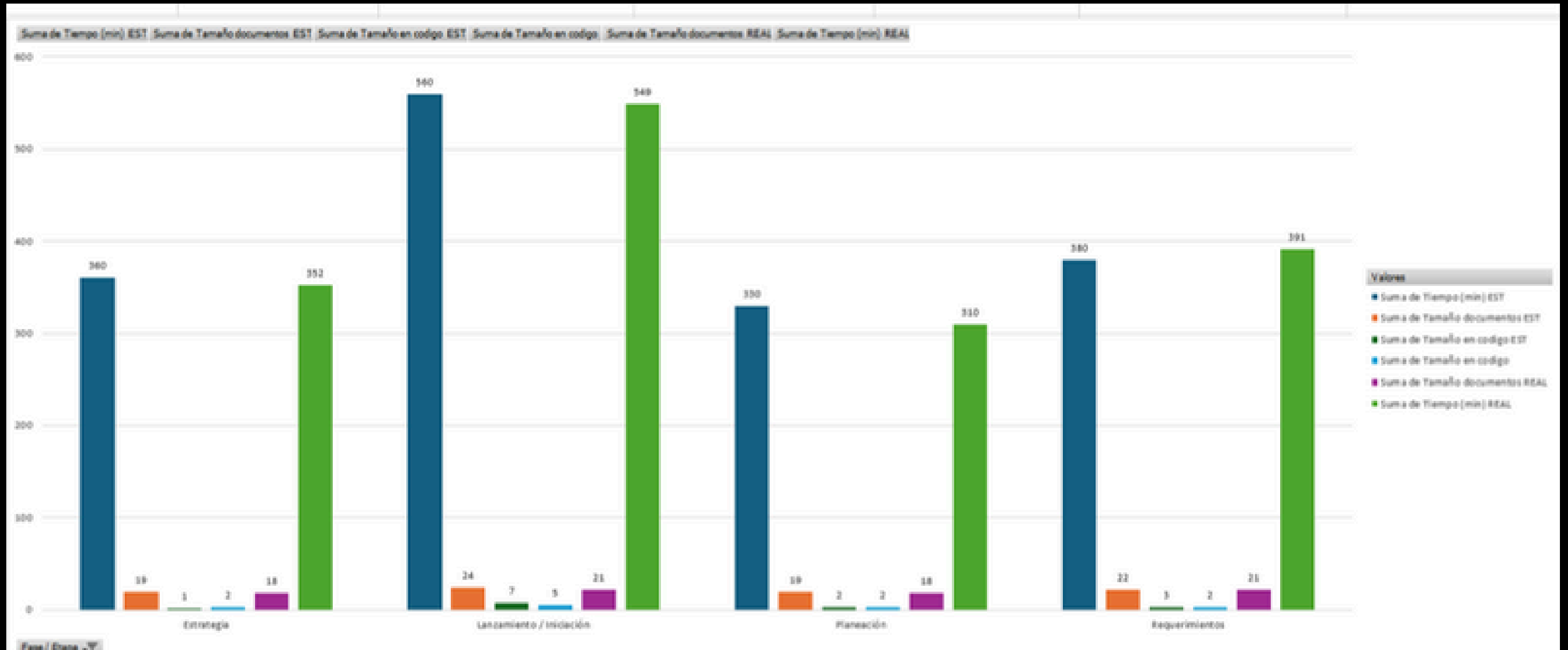
- El Plan de Administración de Requerimientos para el proyecto DashKPI define cómo gestionar, documentar y evaluar los requerimientos del sistema a lo largo de su ciclo de vida. Se establecen roles clave, como los Líderes de Equipo, Calidad, Desarrollo, Planificación, Soporte y Arquitectura, para asegurar la correcta implementación y revisión de los requerimientos. Además, se presentan plantillas para evaluar tanto los requerimientos como las características del sistema, asegurando que sean claros, relevantes, viables y verificables. El plan también cubre la gestión de cambios y la trazabilidad entre requerimientos, tareas y KPIs, con el objetivo de garantizar que el sistema cumpla con los objetivos del proyecto de manera eficiente y controlada.



EDT



```
print(Null);
```

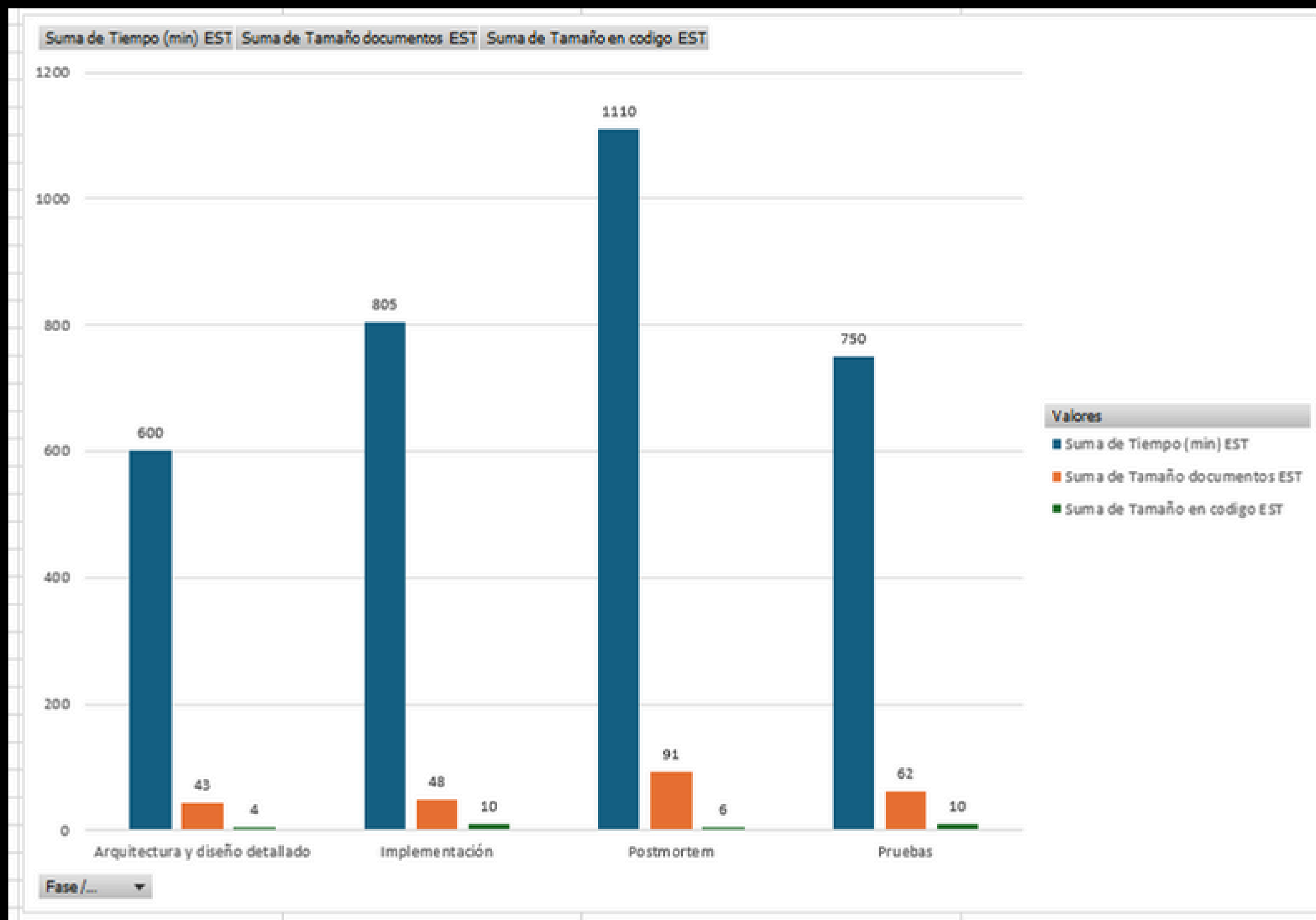




EDT



```
print(Null);
```

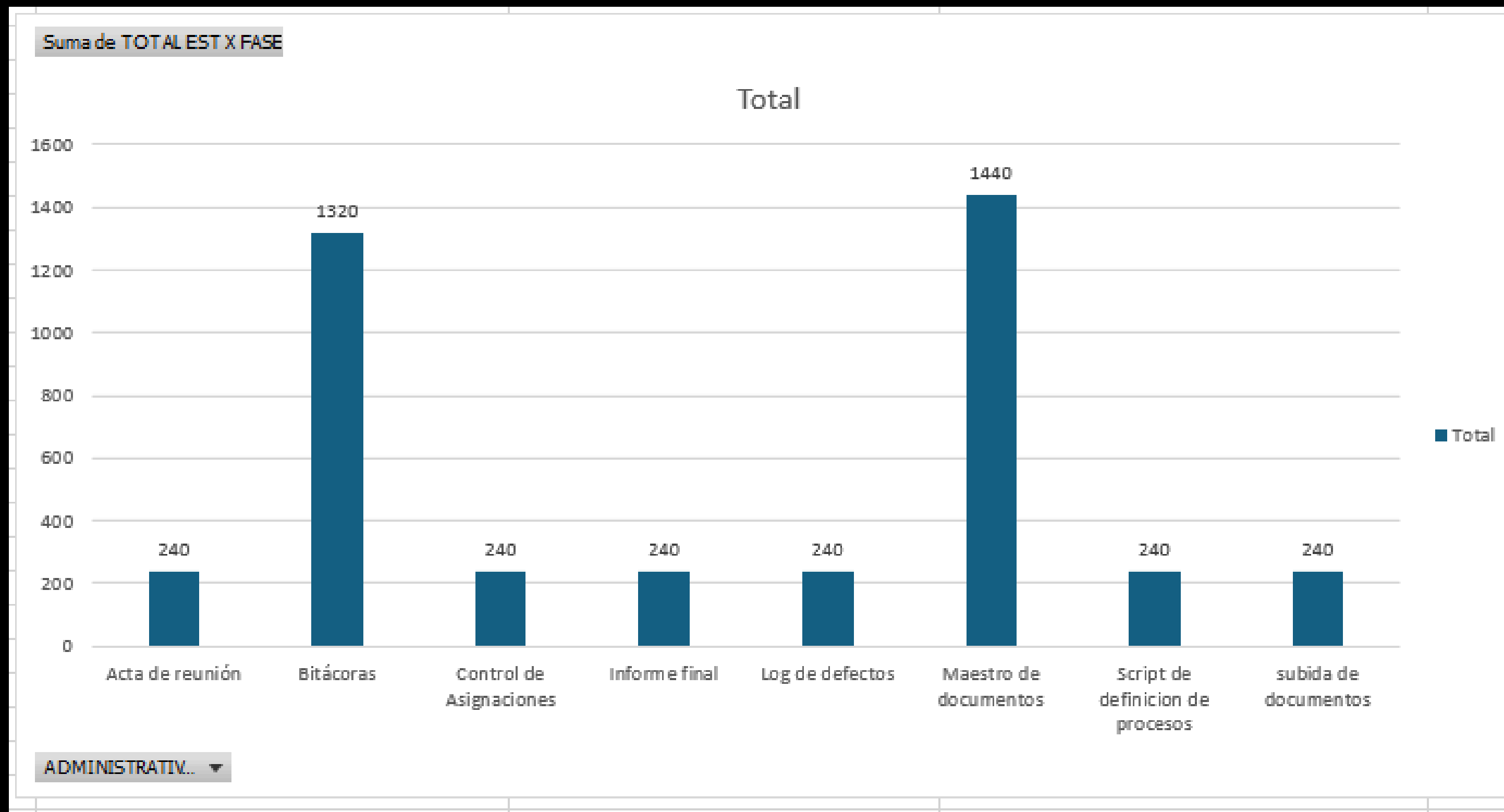




EDT



```
print(Null);
```





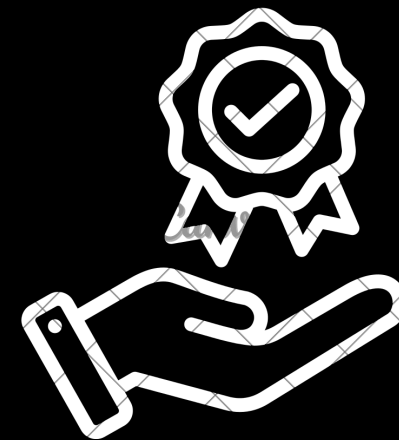
Plan de calidad



El Plan de Calidad de DashKPI establece las políticas, actividades, responsables y métricas que garantizarán la calidad del proyecto en todas sus fases: documentación, desarrollo, pruebas y mantenimiento. El documento incluye los objetivos de calidad, el alcance, los roles definidos, la matriz RACI, las herramientas de soporte, las tareas de calidad (revisiones, inspecciones, registro de defectos y recolección de métricas) y el mecanismo de seguimiento semanal con evidencias y acciones correctivas.

El eje central del plan son las 12 métricas de calidad, asignadas a cada rol clave (Líder de Equipo, Arquitectura, Desarrollo, Soporte, Calidad y Planificación), con fórmulas claras, metas y planes de acción si no se cumplen.

- Métricas seleccionadas
- Líder de Equipo: Porcentaje de revisión, Eliminación de defectos en codificación.
- Líder de Arquitectura: Rendimiento de fase, AFR (prevención vs corrección).
- Líder de Desarrollo: Defectos por KLOC, PDF (Porcentaje Libre de Defectos).
- Líder de Soporte: Inyección de defectos, Eliminación de defectos en soporte.
- Líder de Calidad: Proporción de defectos (Diseño/Unitarias), Rendimiento de proceso.
- Líder de Planificación: Defectos por página, Proporción de tiempos de desarrollo.



Estas métricas permiten medir la eficiencia, confiabilidad y mejora continua del proyecto, asegurando que cada rol aporte al control de calidad con indicadores cuantificables y metas específicas.

En conclusión, el documento servirá como guía integral para asegurar la calidad de DashKPI, enfocándose en el monitoreo constante de métricas y en la toma de acciones correctivas y preventivas para lograr un producto final robusto y confiable.



Log de defectos (Fase 3)

Defectos relevantes resueltos

En el documento del Plan de Administración de Requerimientos, se omitió incluir la parte del checklist, lo cual era importante para verificar que se cumplió con los requisitos establecidos. Este error fue detectado y se solucionó de inmediato.

Defectos relevantes resueltos

En el Plan de Calidad faltaban las métricas específicas para cada integrante del equipo. Este error fue identificado y corregido de inmediato una vez se detectó la omisión.

Defectos relevantes resueltos

En el documento EDT (Estructura de Desglose de Trabajo) faltaba la sección correspondiente a las estimaciones y tiempos necesarios para poder ser más cuantitativos en el plan. Este error fue corregido rápidamente.





OTROS DOCUMENTOS



Control de asignaciones

- ¿Qué es? Matriz para cada actividad con tamaño/tiempo estimado vs. real, fecha/hora de entrega y diferencia.
- Cómo lo usamos: registramos tamaño real (p. ej., hojas/secciones) y tiempo real (min).
- Diferencia = tiempo real – estimado → positivo (retraso), negativo (adelanto).
- Valor: disciplina de seguimiento, evidencias para ajustes de carga y planificación del siguiente ciclo.

Maestro de documentos y SRS

- Es un repositorio centralizado que contiene los enlaces a todos los documentos del proyecto, como casos de uso, atributos de calidad y modelos.

Actas de reunión (Ciclo 1) y checklist

- ¿Qué son? Minutas con temas tratados, acuerdos, responsables y fechas.
- Acuerdos clave: confirmación de roles, backlog inicial y prioridades, criterios de calidad, próxima reunión de control, responsables por documento/artefacto.
- Valor: memoria de decisiones y seguimiento de compromisos.



FASE DISEÑO



ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)



- El presente documento corresponde a la Especificación de Diseño de Software (SDS) del proyecto DashKPI's, desarrollado por el grupo Print(Null) en la Universidad Piloto de Colombia. Su propósito es definir de manera clara y estructurada la arquitectura y el diseño del sistema, proporcionando una guía que oriente la construcción, implementación y validación del software.
- El sistema busca resolver la falta de integración entre tareas, tiempos y KPIs, ofreciendo una plataforma que permita a equipos y líderes de proyecto monitorear el progreso en tiempo real, optimizar la asignación de recursos y garantizar la calidad del producto.





ESPECIFICACIÓN DE DISEÑO DE SOFTWARE (SDS)



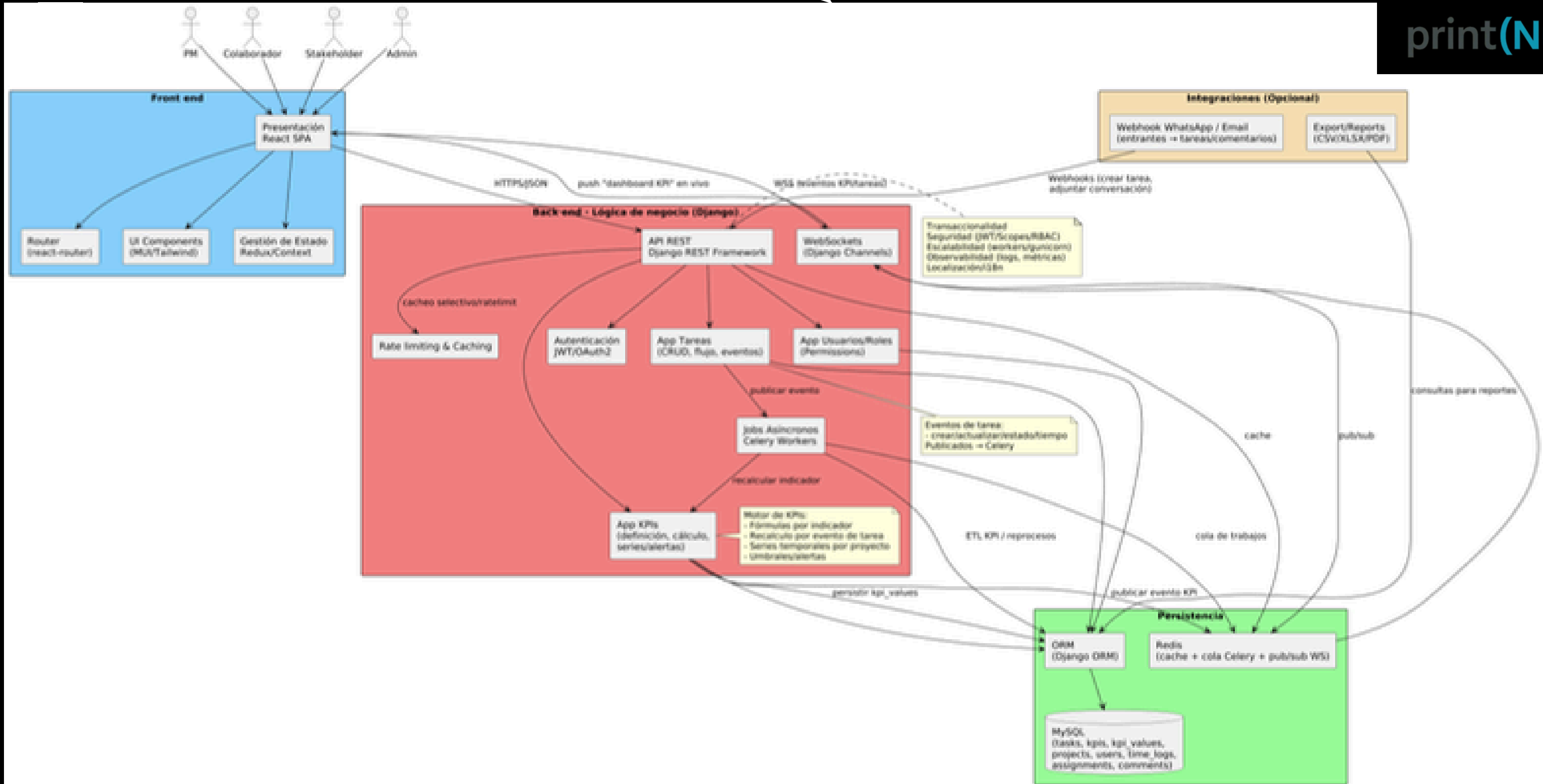
- La definición del problema y los objetivos de la aplicación.
- Las restricciones técnicas y de negocio que guían el desarrollo.
- La arquitectura de software basada en un modelo por capas y orientado a eventos, que separa responsabilidades entre frontend (React), backend (Django) y persistencia (MySQL/Redis).
- Las vistas arquitectónicas UML (contexto, funcional, información, despliegue, desarrollo, concurrencia y operacional) que aseguran consistencia entre los requerimientos y las decisiones de diseño.
- El diseño detallado, incluyendo interfaz de usuario, modelo entidad-relación y modelo de clases.
- Este documento sirve como referencia común para desarrolladores, líderes de rol, clientes y evaluadores académicos, asegurando la trazabilidad de los requerimientos y reduciendo riesgos técnicos en el desarrollo del sistema.



Estilo de arquitectura



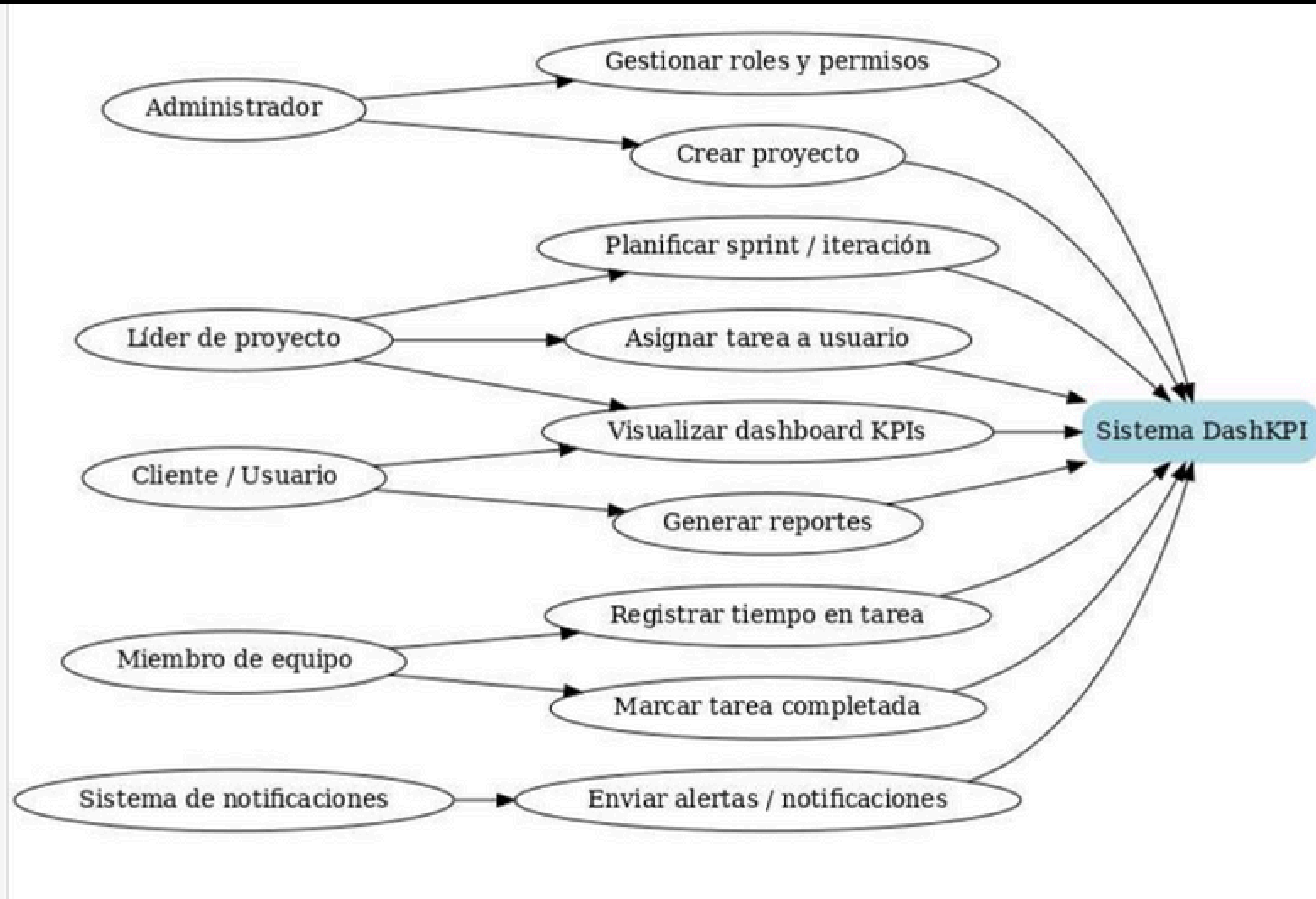
```
print(Null);
```



Vista contexto



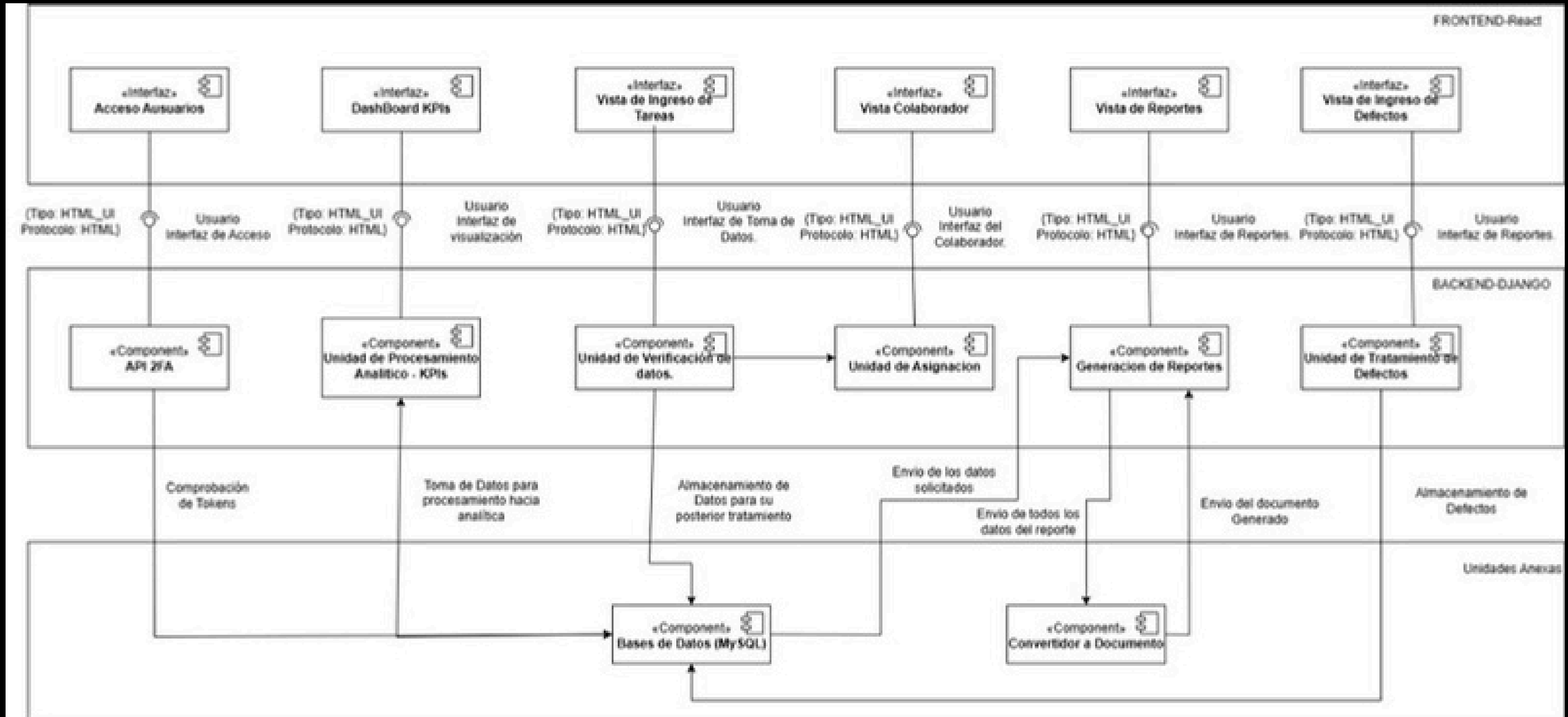
```
print(Null);
```



Vista funcional



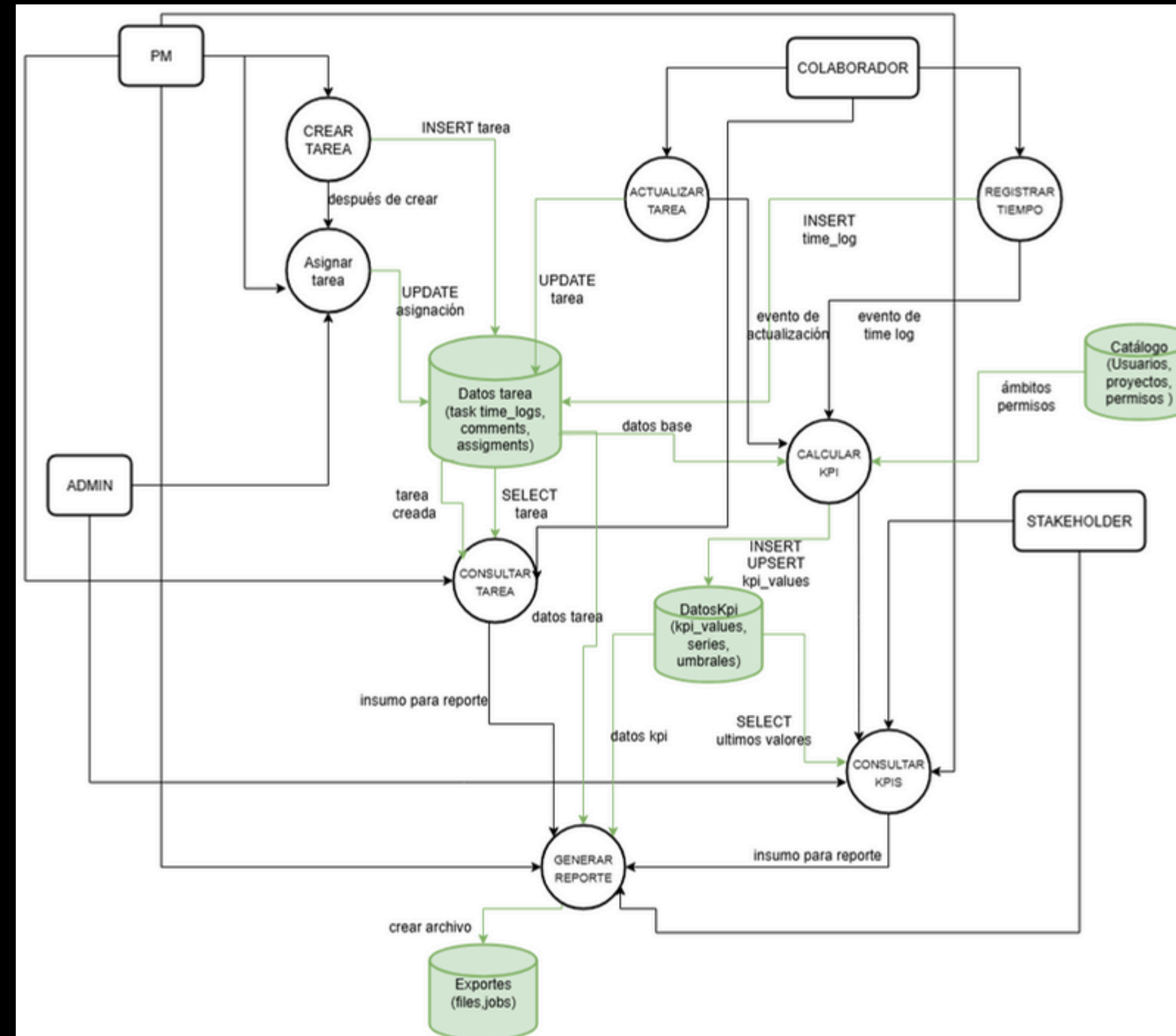
```
print(Null);
```



Vista información



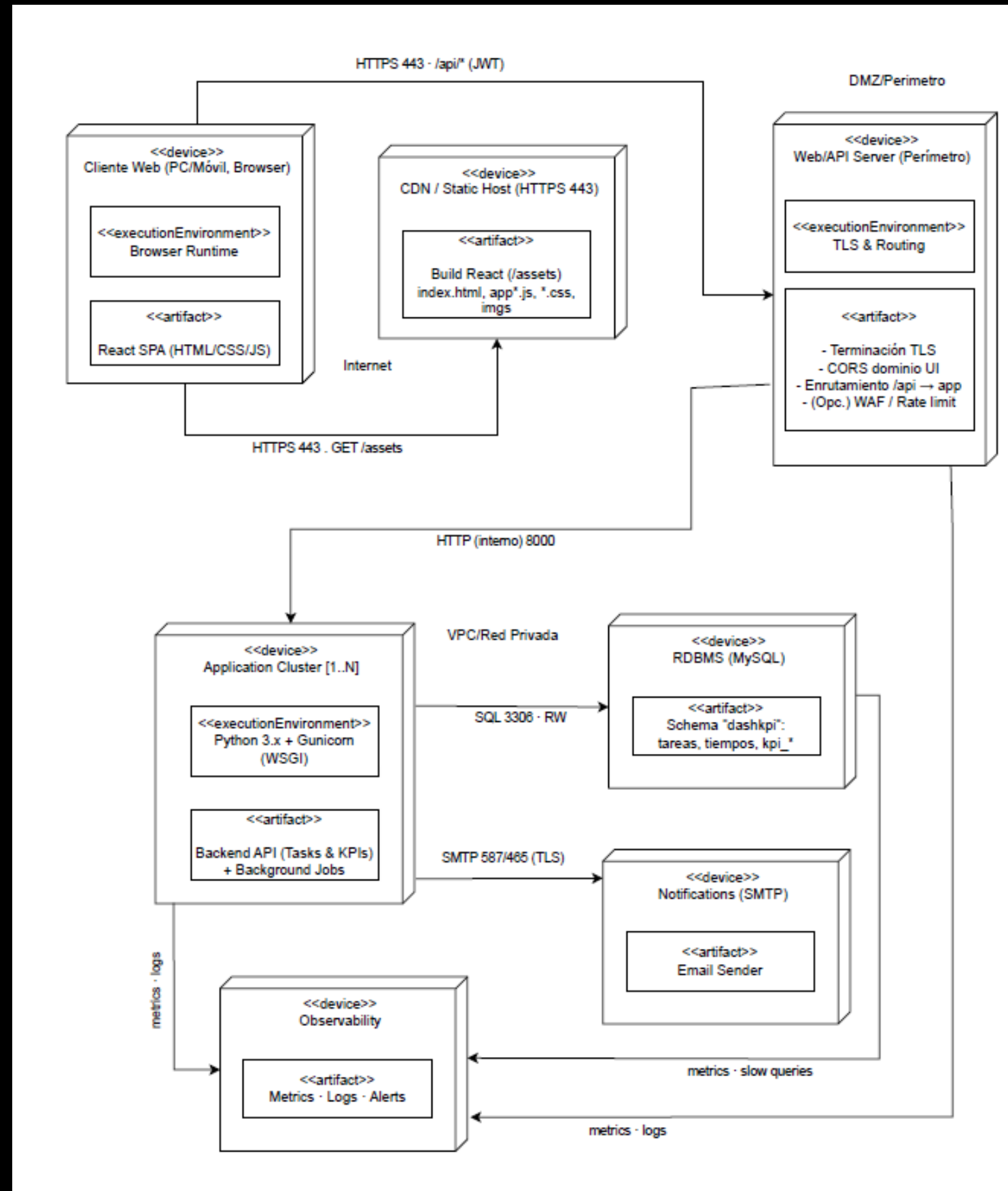
```
print(Null);
```



Vista despliegue



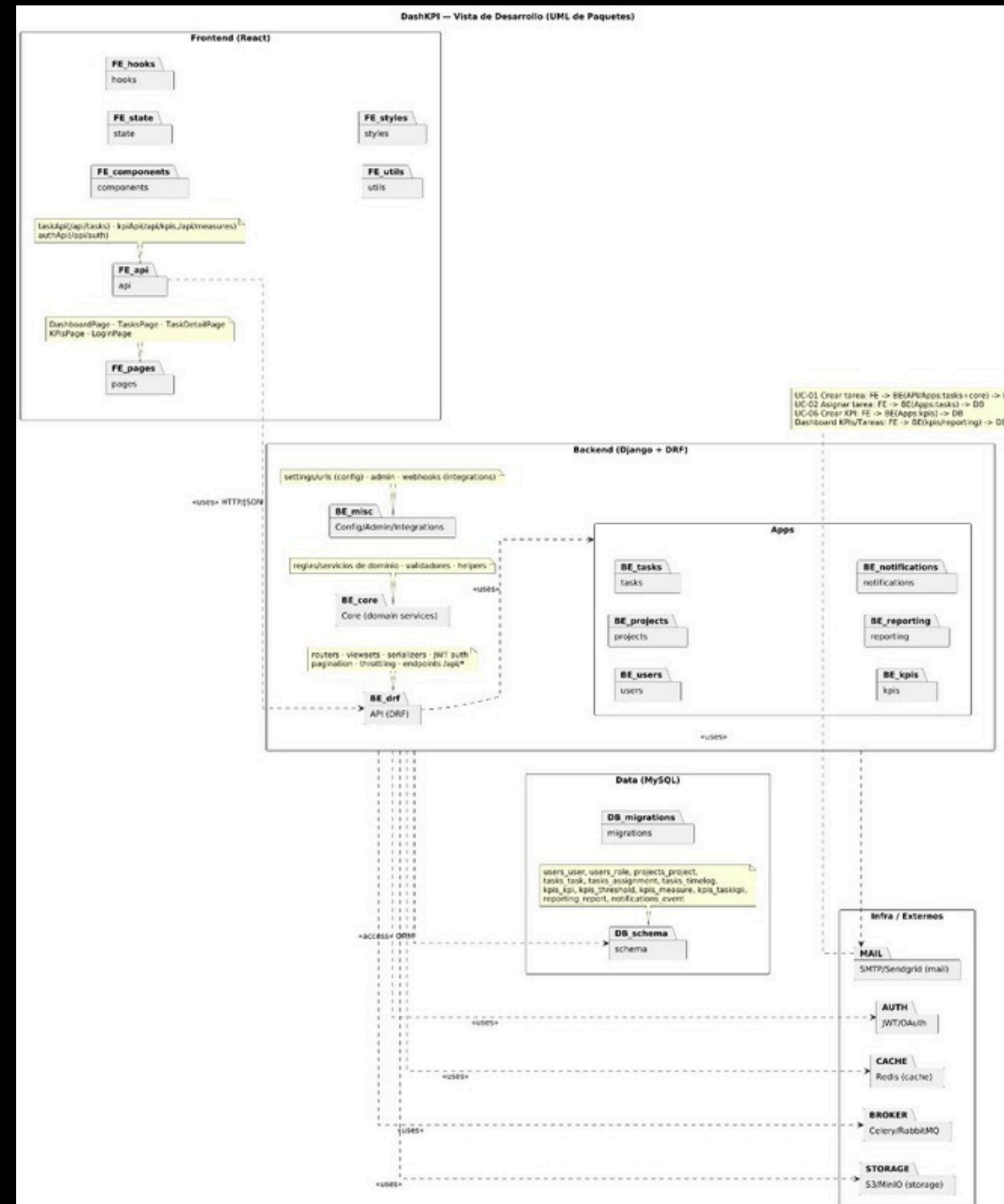
```
print(Null);
```



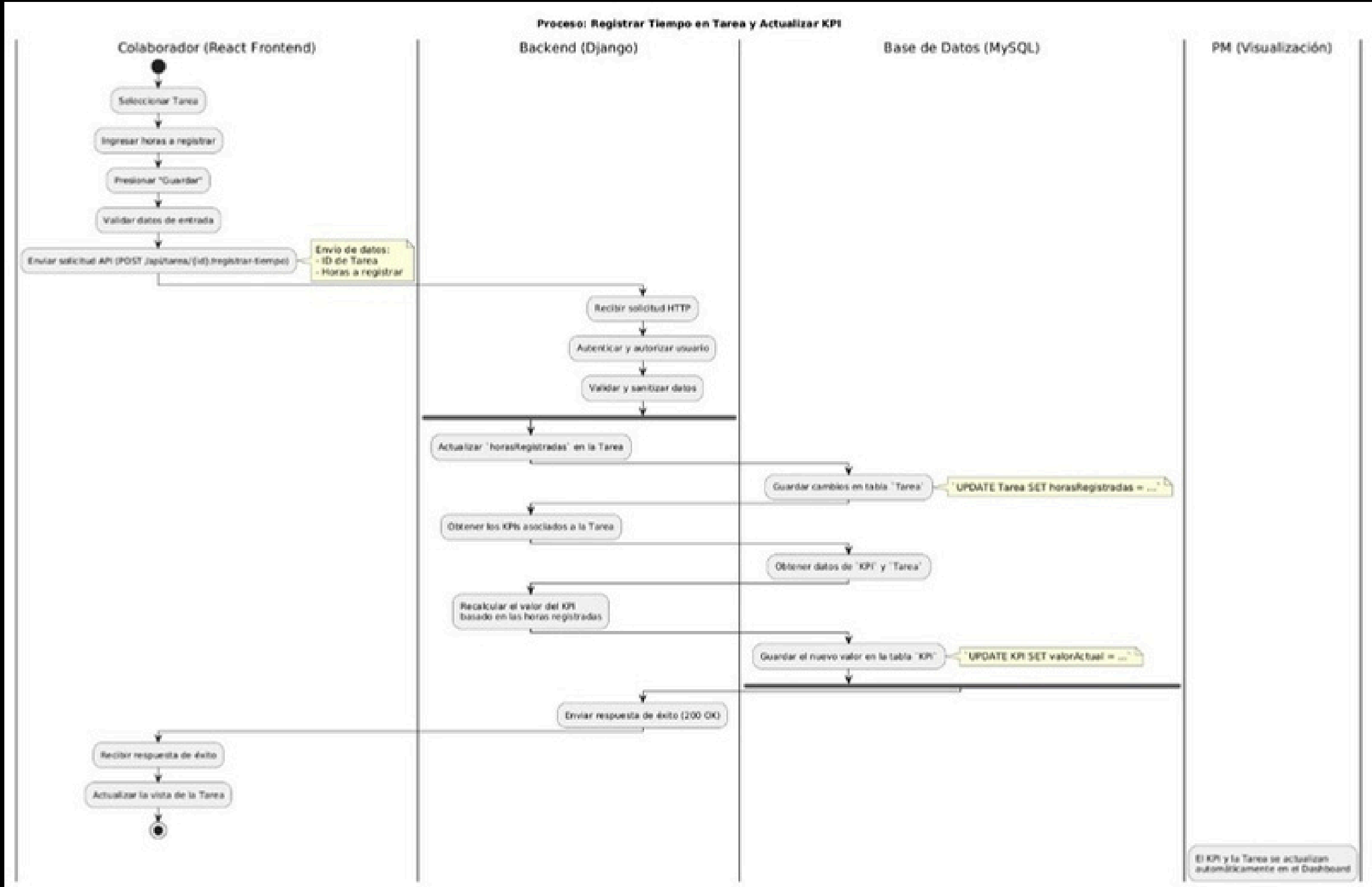
Vista desarrollo



```
print(Null);
```



Vista concurrencia

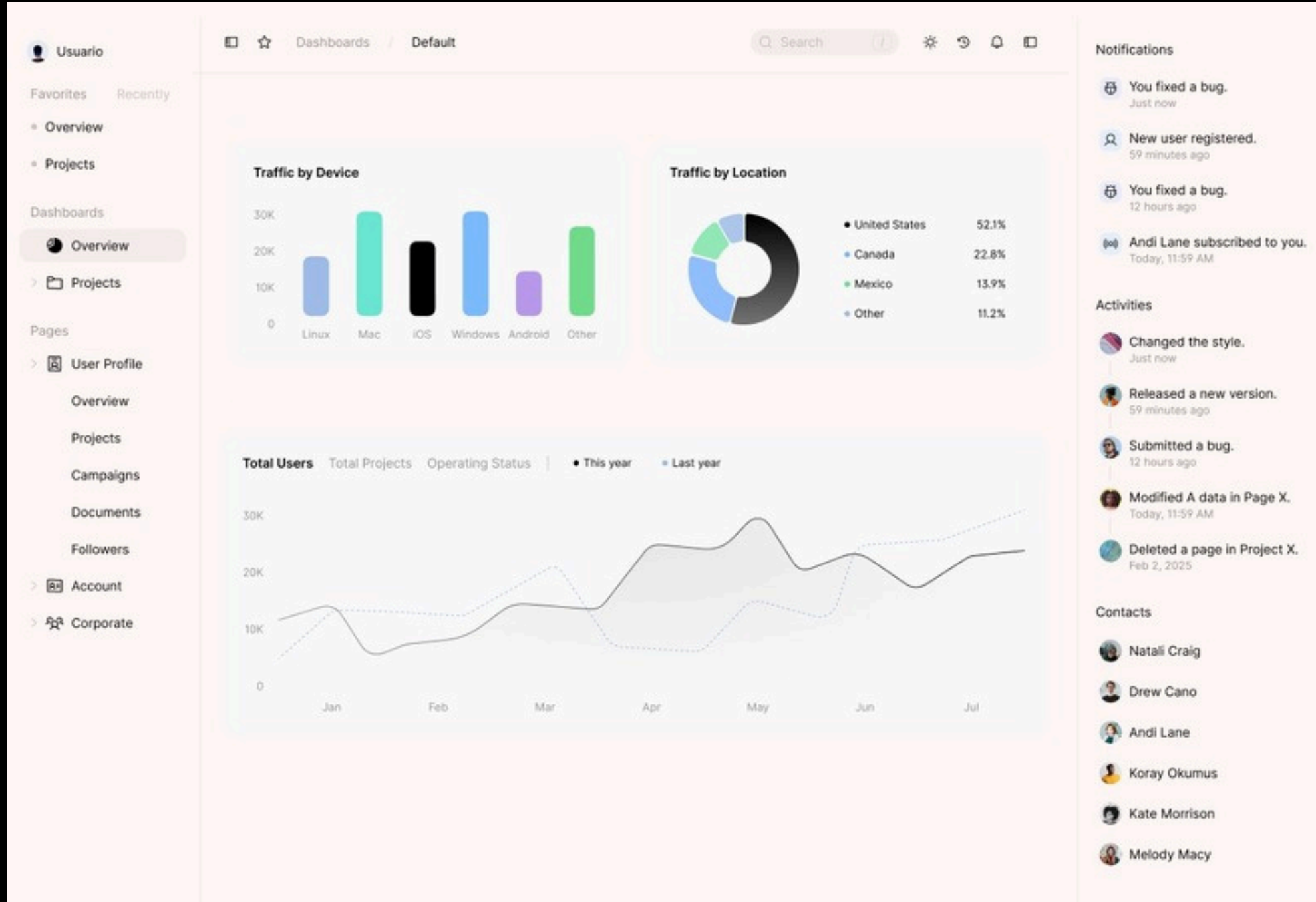


print(Null);

DISEÑO INTERFAZ



print(**Null**);



Start your journey

Sign Up to InsideBox

E-mail

Password

Sign Up

or sign up with

DISEÑO INTERFAZ



```
print(Null);
```

Usuario

Favorites

Recently

Overview

Projects

Dashboards

Overview

Projects

Pages

User Profile

Overview

Projects

Campaigns

Documents

Followers

Account

Corporate

Dashboards

Default

Search

Settings

Refresh

Notifications

Grid

Projects

30 done this month

COMPANIES		MEMBERS	BUDGET	COMPLETION
Soft UI XD Version			\$14,000	
Add Progress Track			\$3,000	
Fix Platform Errors			Not set	
Launch our Mobile App			\$20,500	
Add the New Pricing Page			\$500	
Redesign New Online Shop			\$2,000	

Projects table

PROJECT	BUDGET	STATUS	COMPLETION	ACTION
Spotify	\$2,500	working	60%	
Invision	\$3,000	done	100%	
Jira	\$3,400	cancelled	20%	
Slack	\$1,400	cancelled	0%	
Webdev	\$14,000	working	80%	
Adobe XD	\$2,300	done	100%	

Notifications

You fixed a bug.

Just now

New user registered.

59 minutes ago

You fixed a bug.

12 hours ago

Andi Lane subscribed to you.

Today, 11:59 AM

Activities

Changed the style.

Just now

Released a new version.

59 minutes ago

Submitted a bug.

12 hours ago

Modified A data in Page X.

Today, 11:59 AM

Deleted a page in Project X.

Feb 2, 2025

Contacts

Natali Craig

Drew Cano

Andi Lane

Koray Okumus

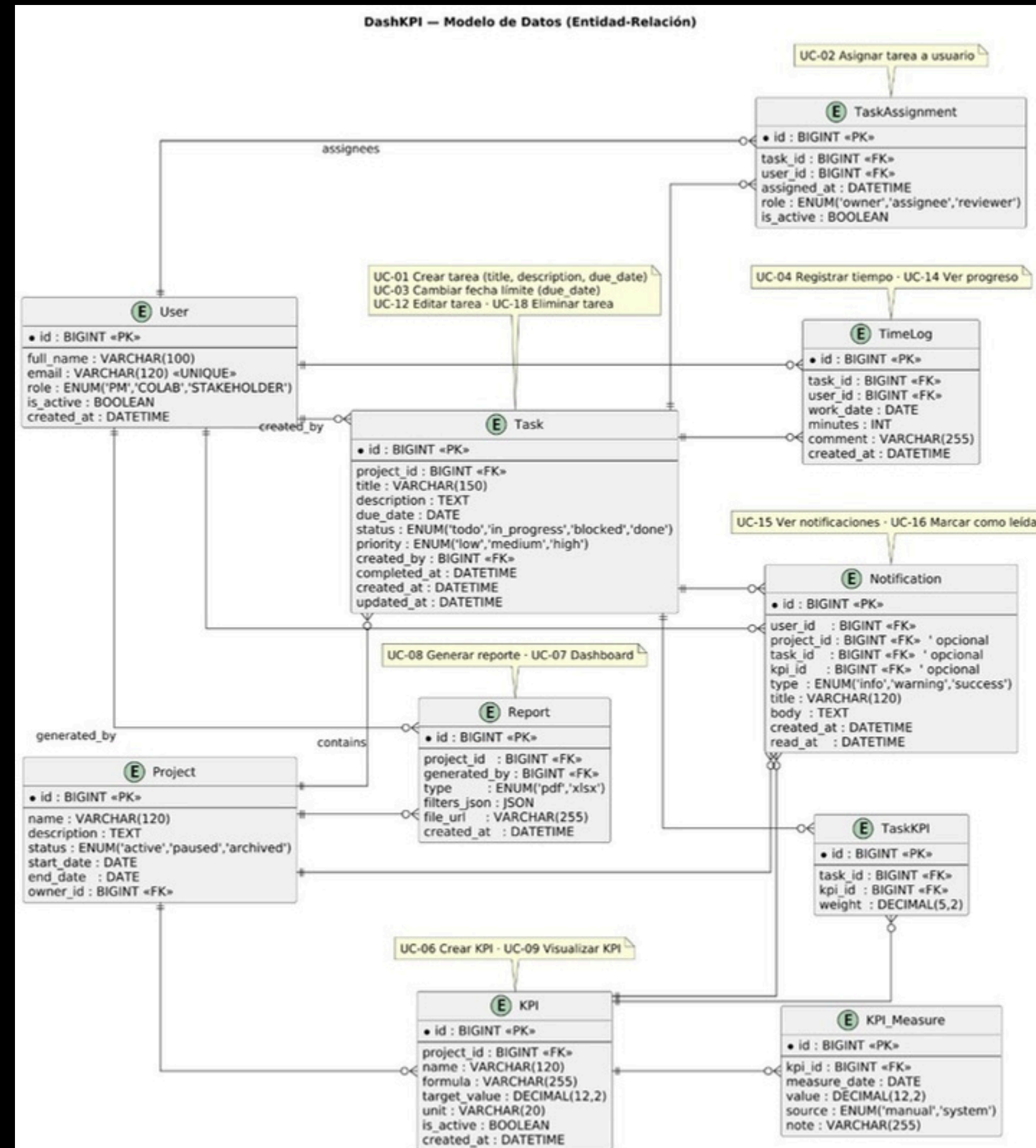
Kate Morrison

Melody Macy

MODELO DE DATOS

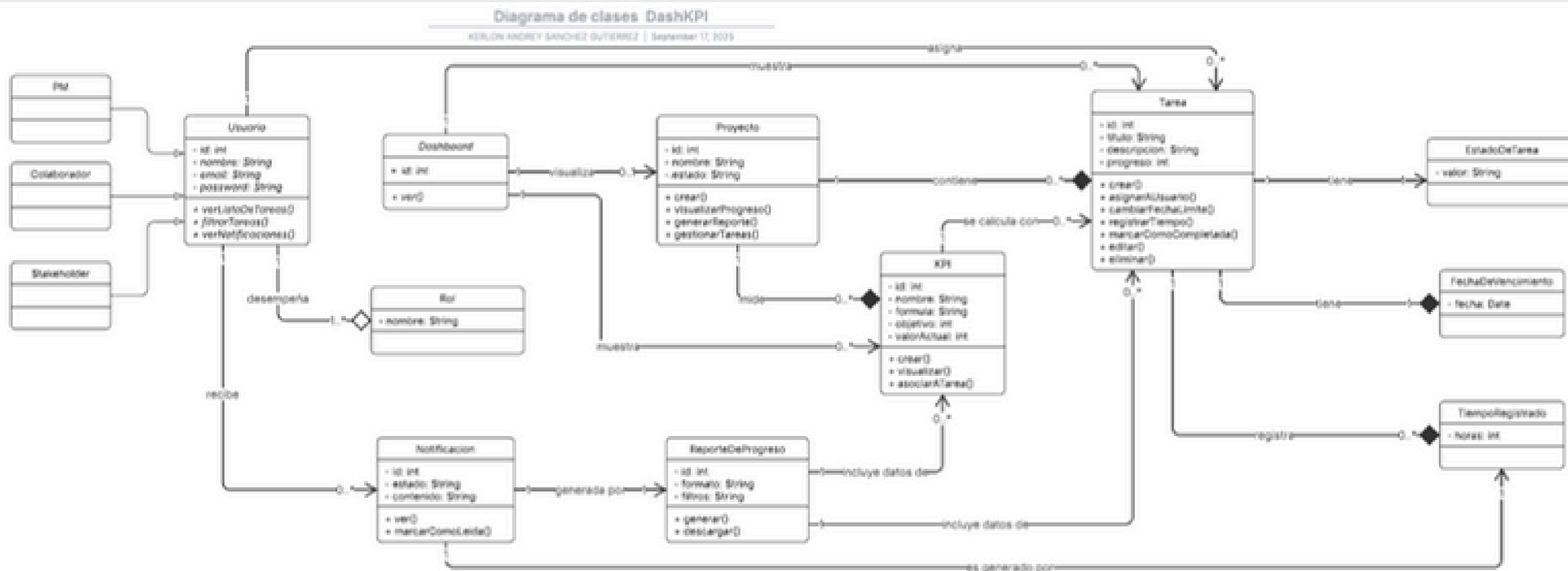


```
print(Null);
```



MODELO DE CLASES

`</>`
`print(Null);`





Log de defectos (Fase 3)

Defectos relevantes resueltos

Durante el desarrollo de la vista funcional, la construcción del diagrama de componentes presentó dificultad porque se generaron varias versiones que no cubrían adecuadamente la funcionalidad. Finalmente, se realizó un ajuste iterativo que permitió obtener un diagrama coherente con los requerimientos.

Defectos relevantes resueltos

El modelo de clases requirió varias versiones porque no satisfacía lo que se buscaba reflejar en el sistema. Tras ajustes sucesivos, se logró estructurar el diagrama de manera que representara de forma precisa las entidades y relaciones del programa





OTROS DOCUMENTOS

Control de asignaciones

- ¿Qué es? Matriz para cada actividad con tamaño/tiempo estimado vs. real, fecha/hora de entrega y diferencia.
- Cómo lo usamos: registramos tamaño real (p. ej., hojas/secciones) y tiempo real (min).
- Diferencia = tiempo real – estimado → positivo (retraso), negativo (adelanto).
- Valor: disciplina de seguimiento, evidencias para ajustes de carga y planificación del siguiente ciclo.

Maestro de documentos

- Es un repositorio centralizado que contiene los enlaces a todos los documentos del proyecto, como casos de uso, atributos de calidad y modelos.

Actas de reunión (Ciclo 1) y checklist

- ¿Qué son? Minutas con temas tratados, acuerdos, responsables y fechas.
- Acuerdos clave: confirmación de roles, backlog inicial y prioridades, criterios de calidad, próxima reunión de control, responsables por documento/artefacto.
- Valor: memoria de decisiones y seguimiento de compromisos.



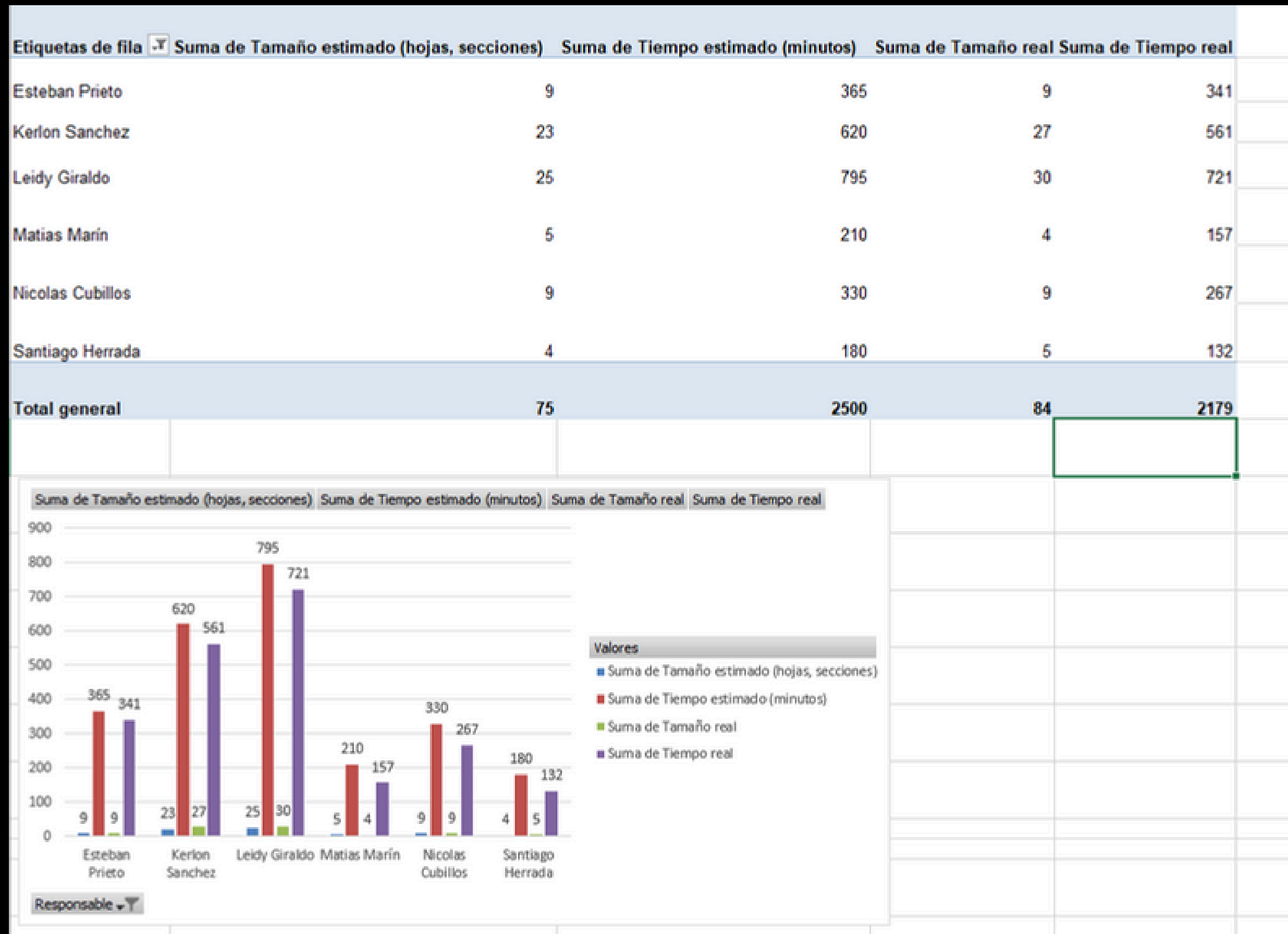
FASE IMPLEMENTACIÓN



CONTROL DE ASIGNACIONES



```
print(Null);
```



Soporte de ejecución



`print(Null);`

- Criterios de completitud del diseño: se cerró el diseño a todos los niveles (sistema→módulo) con arquitectura React + Django + Azure SQL; modelo de datos; contratos API versionados; wireframes y flujos de UI; reglas de negocio; plan de pruebas; seguridad base (contraseñas ≥ 8 con mayúscula y carácter especial, tokens, .env); y aprobación formal antes de codificar.
- Estándares de implementación: se definieron y aplicaron estándares de nombres, interfaces y mensajes (JSON de error unificado), codificación (ESLint/Prettier, PEP8, estructura por apps y servicios), tamaño (LOC y páginas de documentos), defectos (Data/Function/System) y prevención (educación, comunicación, herramientas, supervisión mediante checklist en PR).



Soporte de ejecución



- Estrategia de implementación: enfoque bottom-up (primero modelos/serializers/validadores, luego endpoints y al final pantallas), reutilización (cliente HTTP centralizado, componentes y hooks compartidos, servicios de dominio en BE) y testabilidad (servicios desacoplados, colección Postman/Swagger, datos semilla).
- Revisiones e inspecciones: inspecciones tempranas para atrapar errores aleatorios, revisión de diseño antes de código, flujo de diseño→revisión→utilidades de prueba→inspección→codificación, métricas de eficacia ($\geq 2 \times$ defectos en review vs compilación, > 3 defectos/h, < 200 LOC/h) y reinspección cuando hay dudas. Tablas de ejecución para funciones críticas (login, asignar/completar tarea, validar/cálculo KPI, exportar reporte).



Pruebas Unitarias

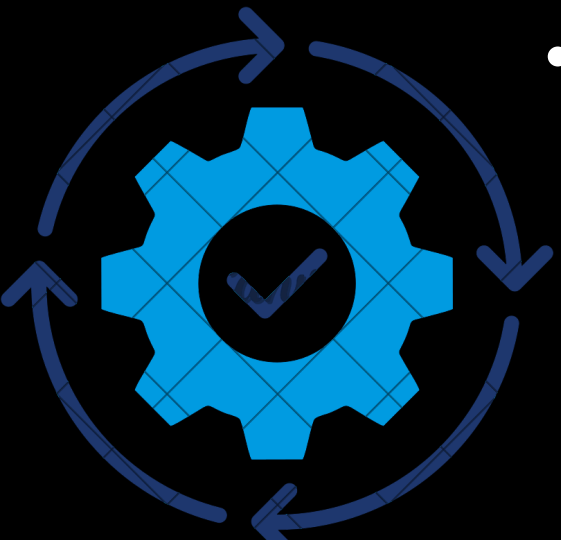


`print(Null);`

- Se analizaron los archivos Login, proyectos y tareas del proyecto.
- A partir del código se construyeron seis tablas (dos por cada archivo), con el objetivo de documentar la lógica de cada vista de la API:
- Tablas de Clase – Método – Parámetro entrada – Resultado
- Identificamos cada clase (vista) y sus métodos (get, post, put, delete).

Registramos los parámetros de entrada que recibe cada vista (por ejemplo: email, password, id_proyecto, id_tarea, progreso).

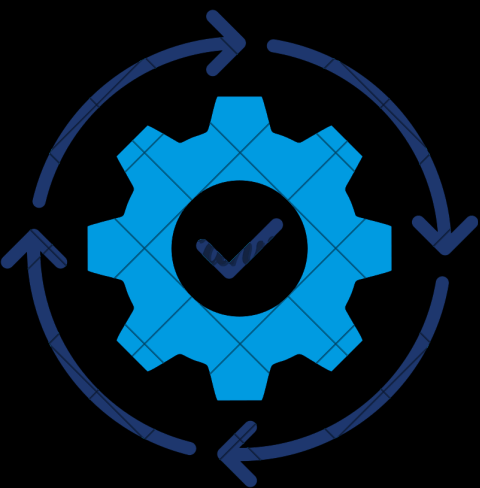
- Resumimos el resultado esperado en cada caso, normalmente el objeto serializado o un código de estado HTTP (200, 201, 204, 400, 401, 404).



Pruebas Unitarias



- Tablas de Clase – Método – Condicional – Entrada – Salida if – Salida else
- Extraímos las validaciones y condiciones presentes en el código (if/else, try/except).
- Mostramos cuál es la entrada que se evalúa (por ejemplo: contraseña correcta, existencia de correo, nombre vacío, formato de fecha, progreso).
- Explicamos las dos posibles salidas: qué ocurre si la condición se cumple (if) y qué ocurre si no se cumple (else).
- De esta forma, el Excel documenta de manera clara:
- Qué hace cada endpoint, Qué datos necesita recibir, Qué devuelve como respuesta, Qué reglas de negocio o validaciones se aplican en cada caso.
- Esto facilita la sustentación, la trazabilidad y las pruebas, ya que se entiende la lógica sin necesidad de leer directamente todo el código fuente.





Log de defectos (Fase 6)

Defectos relevantes resueltos

Por falta de comunicación, un integrante avanzó en la documentación de pruebas unitarias con un enfoque incorrecto. Cuando se revisó en conjunto, fue necesario rehacer el documento completo bajo una estructura consensuada.

Defectos relevantes resueltos

La falta de experiencia inicial en el uso de Azure SQL Server retrasó la implementación de la base de datos común. Tras varias pruebas, se logró sincronizar y estandarizar el acceso para el grupo.

Defectos relevantes resueltos

Algunos integrantes no lograban ejecutar el backend debido a fallos de dependencias y configuraciones iniciales. Fue necesario invertir tiempo en la guía de instalación y validación conjunta.





OTROS DOCUMENTOS



Control de asignaciones

- ¿Qué es? Matriz para cada actividad con tamaño/tiempo estimado vs. real, fecha/hora de entrega y diferencia.
- Cómo lo usamos: registramos tamaño real (p. ej., hojas/secciones) y tiempo real (min).
- Diferencia = tiempo real – estimado → positivo (retraso), negativo (adelanto).
- Valor: disciplina de seguimiento, evidencias para ajustes de carga y planificación del siguiente ciclo.

Maestro de documentos

- Es un repositorio centralizado que contiene los enlaces a todos los documentos del proyecto, como casos de uso, atributos de calidad y modelos.

Actas de reunión (Ciclo 1) y checklist

- ¿Qué son? Minutas con temas tratados, acuerdos, responsables y fechas.
- Acuerdos clave: confirmación de roles, backlog inicial y prioridades, criterios de calidad, próxima reunión de control, responsables por documento/artefacto.
- Valor: memoria de decisiones y seguimiento de compromisos.



Sitio web

<https://leidygbeta1.github.io/Print-Null-/>

E-mail

Print(Null)@gmail.com

Número de contacto

3125102263



print(Null);

Gracias